

Linea Rollup Update

1 Executive Summary

1.1 Audit Phase Structure

2 Scope

3 Findings

3.1 Storage Layout Corruption Due to Inheritance Change

Critical

✓ Fixed

3.2 Concerns Regarding Stage 1 Compliance

Medium

✓ Fixed

3.3 Subtleties and Pitfalls of the Pausing Mechanism Changes

Medium

✓ Fixed

3.4 Reentrancy Possibility for Executors of Upgrade

Medium

✓ Fixed

3.5 Shortcomings of the Current Version Numbering Scheme

Minor

✓ Fixed

3.6 Missing `onlyInitializing` Modifier on Internal Initialization Functions

Minor

✓ Fixed

3.7 Consider Making `external` virtual Functions `public`

Minor

✓ Fixed

3.8 Miscellaneous Informational Points

✓ Fixed

Appendix 1 - Files in Scope

A.1.1 Per Phase

A.1.2 SHA-1 Hashes of Files in Scope for the Final Commit

Appendix 2 - Disclosure

A.2.1 Purpose of Reports

A.2.2 Links to Other Web Sites from This Web Site

A.2.3 Timeliness of Content

Date	February 2026
Auditors	Heiko Fisch, Arturo Roura

1 Executive Summary

This report presents the results of our engagement with the **Linea** team, evaluating the evolution of Linea’s smart contract architecture through a multi-phase differential analysis, examining changes from the previous audit baseline ([contract-audit-2025-01-06](#), commit [a83412e247b7d352b905c906271138e97c1ee5a4](#)) through the yield integration milestone (commit [9c16ce37d2dca6e6106cdc83202979c138ee0130](#)).

The review was conducted in January and February 2026.

The examined changes demonstrate a well-executed architectural modernization initiative that enhances code organization, security, and maintainability. The transition from monolithic to modular contract patterns, introduction of expiration-and-cooldown pausing, and implementation of flexible data availability interfaces reflect disciplined engineering practices. These modifications systematically prepare the codebase for future scalability requirements while maintaining operational integrity and strengthening the overall security posture.

Update February 18, 2026: After completion of the audit, the Linea team has implemented fixes for the points raised and provided an updated version of the code at commit [08622461a7de6160250c2eadc94efd47fd3a5a8d](#) . This version addresses all findings from this report. Moreover, deployment artifacts were added with this commit in directory [contracts/deployments/bytecode/2026-02-17/](#) , and we confirm that the bytecode within these matches the bytecode in the deployment artifacts we built locally.

Update March 27, 2026: Moreover, we confirm that the bytecodes of the contracts deployed at the following addresses match the bytecodes in directory [contracts/deployments/bytecode/2026-02-17/](#) at commit [08622461a7de6160250c2eadc94efd47fd3a5a8d](#) .

- Ethereum:
 - LineaRollup: [0xE68697690E8ff196A6aBB3E1385156D87Df85332](#)
 - TokenBridge: [0xF0e003F0dE2d583Ae28FA8cBF66aa096CdAce3ff](#)
- Linea:
 - L2MessageService: [0x9976fD7edDb78156a002DE74c9158E884702273d](#)
 - TokenBridge: [0x4a496167F187A97379e763f693A499cE1182848b](#)

1.1 Audit Phase Structure

Our audit evaluated changes across four distinct phases, where each phase represents the delta between two specific code states with a defined scope and focus.

Phase 1: Contract Folder Restructure

`contract-audit-2025-01-06` → `contract-folder-restructure`

Focus: Pure organizational refactoring - file relocations from flat structure to hierarchical organization (no functional changes).

Phase 2: Core Feature Integration

`contract-folder-restructure` → `main`

Focus: Multi-faceted protocol enhancements spanning governance, architecture, and optimizations.

Key Changes:

- Expiration-and-cooldown pausing:** Pauses not initiated by Security Council have a timeout and cooldown period (required for stage 1 status)
- Inheritance restructuring:** Monolithic contracts split into modular base classes for better stack consumer extensibility
- Reinitialization cleanup:** Removed previous reinitializer functions for next release
- Solidity 0.8.30 upgrade:** Some floating pragmas remain outside deployment-level contracts
- Transient storage migration:** Replaced custom assembly with native `transient` keyword (EIP-1153) for gas optimization and code size reduction
- TokenBridge deployment simplification:** Expected precomputation of remote bridge addresses (applies to new stack users)
- Visibility changes:** Deprecated fields' visibility reduced for code size and cognitive load reduction
- L2 Prague compiler:** L2 contracts updated to use transient storage in TokenBridge and L2MessageService
- General improvements:** NatSpec cleanup, initialization sequence reordering per SolHint suggestions

Phase 3: Dynamic Chain Configuration

`main` → `feat/996-dynamic-chain-variables`

Focus: Addition of `_verifierChainConfiguration` parameter to rollup contracts for runtime configuration flexibility.

Phase 4: Data Availability Integration

Source: PR #1743 (Modularize data availability submission design)

Focus: Modular data availability layer supporting multiple submission methods.

2 Scope

Given the sequential nature of this multi-phase audit approach, each phase encompasses a distinct set of files corresponding to the specific changes introduced at that development milestone. The scope progressively expands as new components are integrated, reflecting the iterative development process.

The core of the audit concludes with commit 9c16ce37d2dca6e6106cdc83202979c138ee0130 from PR #2033 (Integrate yield), which represents the final integration milestone encompassing all previous phases before fixes.

While this PR integrates the Yield Manager contracts, some of these components fall outside of this engagement scope and were previously audited under the dedicated Linea Yield Manager engagement.

The detailed list of files in scope for each phase can be found in the Appendix.

3 Findings

Each issue has an assigned severity:

- Critical** issues are directly exploitable security vulnerabilities that need to be fixed.
- Major** issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Medium** issues are objective in nature but are not security vulnerabilities. These should be addressed unless there is a clear reason not to.
- Minor** issues are subjective in nature. They are typically suggestions around best practices or readability. Code maintainers should use their own judgment as to whether to address such issues.
- Issues without a severity are general recommendations or optional improvements. They are not related to security or correctness and may be addressed at the discretion of the maintainer.

3.1 Storage Layout Corruption Due to Inheritance Change **Critical** **✓ Fixed**

Resolution
Fixed as recommended in commit 08622461a7de6160250c2eadc94efd47fd3a5a8d.

Description

The previous version of the TokenBridge contract has been refactored into a TokenBridgeBase abstract contract that contains almost all of the functionality of the original TokenBridge and a new, minimal TokenBridge contract which essentially only inherits from TokenBridgeBase and provides an initialize function.

As part of the changes, OpenZeppelin’s ReentrancyGuardUpgradeable has been replaced with a self-written TransientStorageReentrancyGuardUpgradeable, which uses transient storage to store whether the contract has already been entered or not. Crucially and correctly, TransientStorageReentrancyGuardUpgradeable has a gap variable of size 50, which equals the number of storage slots ReentrancyGuardUpgradeable occupies (including its gap). However, ReentrancyGuardUpgradeable inherits from Initializable – which also uses storage – but TransientStorageReentrancyGuardUpgradeable doesn’t. On the other hand, AccessControlUpgradeable, which is the next contract TokenBridgeBase inherits from, also inherits from Initializable, so Initializable doesn’t disappear entirely, but it moves to a different place in the C3 linearization.

This corrupts the storage layout. Specifically, the new storage slots utilized by Initializable used to be in ReentrancyGuardUpgradeable’s gap – making the contract appear uninitialized and allowing anyone to call initialize.

A similar change has been made in the L2 messaging contracts. More specifically, in L2MessageServiceV1’s list of inherited contracts, ReentrancyGuardUpgradeable has been replaced with TransientStorageReentrancyGuardUpgradeable, but since more base-like contracts in this list inherit from Initializable too, C3 linearization and storage layout don’t change.

However, in both cases, the storage previously used by ReentrancyGuardUpgradeable remains “dirty”. (The value 1 is used to indicate “not entered”, not 0.) As this storage slot moves into TransientStorageReentrancyGuardUpgradeable’s gap, it is not used anymore, but if the gap is ever reduced in size and the corresponding storage slot is made available for regular usage again, the expectation will probably be that it is zero. Hence, it is advisable to clean the storage slot during the upgrade.

Recommendation

- In TokenBridgeBase, insert Initializable in the list of inherited contracts, between ITokenBridge and TransientStorageReentrancyGuardUpgradeable. This leaves the C3 linearization and the storage layout unchanged.
- In TokenBridge, add a reinitializeV3 function (with a reinitializer(3) modifier) that sets the storage slot previously used by ReentrancyGuardUpgradeable to zero.
- In L2MessageService, add a reinitializeV3 function (with a reinitializer(3) modifier) that sets the storage slot previously used by ReentrancyGuardUpgradeable to zero.

In 2 and 3 above, the new reinitialization function either has to be restricted to the proxy admin, or the initializer modifier for the initialize function has to be replaced with reinitializer(3) (to protect new deployments). We discuss these options in more detail in issue 3.5. Either way, upgrade and reinitialization should always happen atomically, as should deployment and initialization.

Remark

This is a self-reported finding by the client that was communicated to us at the beginning of the engagement.

3.2 Concerns Regarding Stage 1 Compliance Medium ✓ Fixed

Resolution
With the concerns raised here in mind, the client has revised the pausing mechanism. The new implementation in commit 08622461a7de6160250c2eadc94efd47fd3a5a8d addresses all pausing-related points we raised in this report (this finding and the next one, specifically). Edge cases to be mindful of have been discussed with the Linea team.

Introduction

The upgrade of the contracts includes an enhanced pausing mechanism with the goal to keep actors that enjoy less trust than the Security Council from pausing the contracts indefinitely. Essentially, this is achieved via a timeout/expiry and a cooldown period: Unless the Security Council intervenes before that, after expiry, anyone can unpause, and it is not possible for less trusted actors than the Security Council to start a new pause during the cooldown period.

We briefly summarize some key aspects of the pausing mechanism that play an important role in this finding and the next:

- There are several different pause types, which can be set and unset independently of each other. Each type affects a different functionality of the protocol.
- Expiry and cooldown are shared for all pause types.
- If the Security Council (SC) sets a pause, the expiry will be set to infinity (technically, `type(uint256).max - COOLDOWN_DURATION`).
- If an actor with less trust than the Security Council (non-SC) sets a pause, the expiry will be set to `block.timestamp + PAUSE_DURATION ; PAUSE_DURATION` is set to 3 days. After expiry, anyone can unset any pause type. New pauses can't be set until the cooldown period is over (except by the SC); `COOLDOWN_DURATION` is 1 day.
- If the SC unsets a pause, the timeout will be set to `block.timestamp - COOLDOWN_DURATION`. Hence, even non-SC actors can set new pauses right away.
- If a non-SC actor unsets a pause, the currently set expiry doesn't change.

Description

The expiration-and-cooldown mechanism [suggested by L2Beat](#) for emergency pauses is fairly straightforward as long as there is only a single pause type, but things becomes more tricky when several pause types are involved, as in our case. Specifically, consider the following situation:

There is a malicious non-SC actor Eve with permission to pause type X. Eve can force the SC into inactivity, e.g. by bribing a minority. Eve waits patiently until the SC sets (or extends) some pause type Y different from X. She frontruns this transaction and pauses type X. The SC's action will set expiry to inifinity, and as there is only one global expiry, pause type X will not expire.

It is – perhaps – debatable whether this behavior is sufficient for stage 1. While it is true that an indefinite pause has been set by the SC, this was a pause of type Y; an indefinite pause of type X was certainly not intended by the SC in this scenario, only by Eve. And actors like Eve should not have the ability to set indefinite pauses (probably not even through tricks like frontrunning a SC-initiated pause of a different type).

Recommendation

We recommend avoiding such scenarios and playing it safe instead. Specifically, a separate function – only callable by the SC – should be implemented which takes as input an array that, for every pause type, contains a boolean value indicating whether this pause type should be set or not. Of course, this function should then set all pauses accordingly. If the array contains only zeros (i.e., everything should be unpaused), expiry should be set to the past (`pauseExpiryTimestamp = block.timestamp - COOLDOWN_DURATION;`), i.e., we return to normal operation; otherwise, expiry should be set to infinity (`pauseExpiryTimestamp = type(uint256).max - COOLDOWN_DURATION;`). All SC-related logic that was introduced in `pauseByType` and `unpauseByType` should be removed again.

Admittedly, this is a bit cumbersome, but the advantage of this approach is that the entire pausing state – across all pause types – is explicitly approved by the SC.

3.3 Subtleties and Pitfalls of the Pausing Mechanism Changes Medium ✓ Fixed

Resolution
With the concerns raised here in mind, the client has revised the pausing mechanism. The new implementation in commit 08622461a7de6160250c2eadc94efd47fd3a5a8d addresses all pausing-related points we raised in this report (this finding and the previous one, specifically). Edge cases to be mindful of have been discussed with the Linea team.

Description

A brief summary of the pausing mechanism has been given in the introduction of the previous finding. In addition to the concerns mentioned there, it has a few pitfalls and subtleties, which we will discuss now:

A. Once a certain pause type is set, this prevents other pauses from being set too (unless by the SC). This is known; in fact, there is a comment in the code mentioning this. Nevertheless, we think the situation is at least not ideal. Assume, for example, there is an attack scenario that could be quickly stopped by setting pause type X. The attacker, aware of this fact, then has an incentive to bribe a non-SC actor with the right permissions to set some other pause type Y and would start the attack as soon as that happened. As mentioned above, pause type X can now only be set by the SC – which might require significantly more coordination and, therefore, time than other actors with the permission to set pause type X would need for pausing. Hence, this change could negatively impact the speed with which functionalities can be paused in some selected/provoked situations.

B. Assume several pause types have been set. As soon as the SC unsets one of these, all other pauses can immediately be unset by *anyone*, which might very well be unintended (and might even go unnoticed). There is a workaround: Assume pause types X and Y have been set, and the SC wants to unpause X, but Y should remain paused. This can be achieved by atomically unpausing X, unpausing Y, and repausing Y. Still, it doesn’t seem far-fetched that this quirk and the necessity for the workaround will be forgotten in difficult situations, when the SC is under pressure to resolve a situation.

C. Assume a certain pause type has been set by a non-SC actor. After the timeout (3 days), anyone will be able to unset the pause. The SC reviews the situation and comes to the conclusion that setting this pause was warranted and would, in fact, like to extend it because the situation won’t be resolved before the timeout. However, there is no direct way for the SC to extend a pause; if it’s already set, it can’t be set again before it’s been unset. Again, there is the workaround to atomically unset and reset the pause, but that is cumbersome, to say the least.

Similarly, if a non-SC actor sets a pause, and then the same or a different non-SC actor quickly comes to the conclusion that this was a false alarm and unsets the pause, then expiry and cooldown will remain. The SC would like to step in and reset everything to the original state, so that non-SC actors can pause if they deem it necessary, but again, there is no direct way for the SC to do that and a workaround like atomically setting and unsetting some pause is necessary if waiting out the cooldown is not acceptable.

These two situations are less problematic than the scenarios described in A and B because there is no direct risk involved. Still, the workarounds can be a nuisance and increase the coordination effort in the SC.

Recommendation

The changes recommended in [issue 3.2](#) also resolve B and C.

To address A, consider implementing the following change: Non-SC actors can set additional pause types as long as expiry is in the future (but not infinity); expiry should *not* change through this action.

3.4 Reentrancy Possibility for Executors of Upgrade Medium ✓ Fixed

Resolution
Fixed as recommended in commit 08622461a7de6160250c2eadc94efd47fd3a5a8d .

Description

As already discussed in [issue 3.1](#), the contracts that were using OpenZeppelin’s `ReentrancyGuardUpgradeable` had it replaced with `TransientStorageReentrancyGuardUpgradeable`. In such a case, an actor who can execute an update – if the proxy admin owner is a timelock, that’s any address with the `EXECUTOR_ROLE` – can launch a “cross-upgrade reentrancy attack” by first exiting through the old implementation, executing the upgrade, and then reentering with the new implementation in place. The new reentrancy guard will check the transient storage and ignore the “entered” state we set in the regular storage with the old reentrancy guard (even assuming it didn’t get reset during the upgrade, as recommended in [issue 3.1](#)).

How much of a problem this is depends on the configuration of the `EXECUTOR_ROLE`. It is possible to configure a timelock such that *anyone* can execute; in this case, anyone could launch a reentrancy attack as described above. The risk is lower, obviously, if execution is restricted to a highly trusted role. But even in this case, this should better be prevented on a technical level than delegated to trust.

While `ReentrancyGuardUpgradeable` was replaced with `TransientStorageReentrancyGuardUpgradeable` in `TokenBridge` / `TokenBridgeBase` and `L2MessageServiceV1`, `L1MessageService` / `L1MessageServiceBase` had already used `TransientStorageReentrancyGuardUpgradeable` before this upgrade. However, `TransientStorageReentrancyGuardUpgradeable` has seen changes in this upgrade; more specifically, the previously used namespaced transient storage – i.e., the “entered” status was stored at a hashing-derived transient storage slot – has been replaced with a “regular” transient storage variable (presumably, because this is now supported by the compiler). Hence, this change leads to a similar situation as above: the location where the “entered status” is stored changes, this time not from regular storage to transient storage but within transient storage, opening up the same reentrancy possibility as above.

Finally, it should be noted that any time in the future a new transient storage variable is introduced somewhere and it lands before the `TRANSIENT_ENTERED` slot used by `TransientStorageReentrancyGuardUpgradeable`, the same reentrancy risk pops up again, as this will make the `TRANSIENT_ENTERED` slot move. Hence, we think using a hash-derived slot in `TransientStorageReentrancyGuardUpgradeable` wasn’t a bad idea after all.

Recommendation

1. In [issue 3.1](#), we recommended setting the storage slot that was previously used by `ReentrancyGuardUpgradeable` to zero in the `TokenBridge` and `L2MessageService` reinitialization functions. Before doing that, add a check that the storage slot has indeed the “not entered” value `1` – and revert if not. That prevents reentrant upgrading of these contracts.
2. In `TransientStorageReentrancyGuardUpgradeable`, keep using the namespaced (hash-derived) transient storage slot that was used before. This prevents slot changes now as well as in future upgrades that introduce new transient storage variables.
3. As a general rule (and assuming the storage used by the reentrancy guard doesn’t change), it seems sensible to add a `nonReentrant` modifier to reinitialization functions if a reentrant upgrade can’t be ruled out with certainty otherwise. The reason is that even if the reentrancy guard works reliably “across the upgrade”, there could still be a reentrancy vulnerability that doesn’t exist within the old or within the new implementation, but only “across the old and new implementation”. In that spirit, we recommend adding a `nonReentrant` modifier to `reinitializeV8` in `LineaRollup`.

3.5 Shortcomings of the Current Version Numbering Scheme Minor ✓ Fixed

Resolution
Fixed as recommended in commit 08622461a7de6160250c2eadc94efd47fd3a5a8d .

Description

Upgradeable contracts that employ OpenZeppelin’s framework typically inherit from `Initializable` , which basically provides a version number (stored in `_initialized`) and an `initializer` as well as a `reinitializer(x)` modifier – where the former sets the version number to `1` , the latter to `x` – and both ensure that the new version number is strictly greater than the current one.

Deployable contracts in the Linea codebase usually have an `initialize` as well as a `reinitializeVx` function, where `x` is a concrete number. The presence of both functions allows fresh deployments as well as upgrades to the new version. To simplify the discussion, we will focus the rest of the discussion on the `LineaRollup` contract, but the same principle applies to other contracts, too.

`LineaRollup` has an `initialize` function with an `initializer` modifier and a `reinitializeV8` function with a `reinitializer(8)` modifier. This scheme has the following (minor) drawbacks and inelegancies:

- It is necessary to introduce access control on the `reinitializeV8` function and restrict it to the proxy admin; otherwise, anyone could call this function on new deployments. That exposes an implementation detail of the proxy on the implementation contract.
- The proxy admin owner could wrongly upgrade to V8 when the currently used implementation is V6 or lower. For fresh deployments (`_initialized == 1`), the proxy admin owner could even “upgrade” to an earlier version. Of course, we’d expect a highly privileged actor like the proxy admin owner to know what they’re doing and to be extra careful, but it’s still better to eliminate possibilities for mistakes in the first place – if that’s possible.
- If the number stored in the `_initialized` variable is `1` , this doesn’t tell you anything about which version is really in use currently. In particular, this increases the risk to upgrade to a wrong version, as discussed in the previous item.

If we replace the `initializer` modifier with `reinitializer(8)` , the situation changes as follows:

- There is no need for access control and no need to introduce the proxy admin on the implementation contract because the `reinitializeV8` function now fails on new deployments of this version (which already have `_initialized == 8`).
- The `_initialized` variable – retrievable via `_getInitializedVersion` – now tells us which version we’re on, even for new deployments.
- If, in `reinitializeVx` , we verify `_getInitializedVersion() == X-1` , we can reliably prevent upgrades from the wrong version.
- If, in `initialize` , we verify `_getInitializedVersion() == 0` , we can ensure that `initialize` is only called for new deployments, not during upgrades.

Two points to be aware of:

1. For both patterns, deployment+initialization as well as upgrade+reinitialization have to be executed atomically.
2. The `_getInitializedVersion() == X-1` check mentioned above can’t yet be introduced when the switch to this pattern is made. The reason is that there might still be fresh V7 deployments (with `_initialized == 1` , according to the current pattern) that should be upgradeable to V8. Only in V9 should this check be added then.

Recommendation

Consider adopting the pattern discussed above across the codebase.

3.6 Missing `onlyInitializing` Modifier on Internal Initialization Functions Minor ✓ Fixed

Resolution
Fixed as recommended in commit 08622461a7de6160250c2eadc94efd47fd3a5a8d .

Description

A recurring theme in this upgrade is that contracts have been split up. This has led to the introduction of new abstract base contracts, which typically have an internal initialization function. Examples include:

- `__TokenBridge_init` in `TokenBridgeBase` ;
- `__L2MessageService_init` in `L2MessageServiceBase` ;
- `__LineaRollup_init` in `LineaRollupBase` ;
- `__LivenessRecovery_init` in `LivenessRecovery` .

In all these cases, the `onlyInitializing` modifier is missing. While this modifier is not strictly necessary, it is customary and probably considered a best practice to add it to internal initialization functions.

Recommendation

We recommend having an `onlyInitializing` modifier on all internal initialization functions.

Remark

This is a self-reported finding by the client that was communicated to us at the beginning of the engagement.

3.7 Consider Making `external virtual` Functions `public virtual` Minor ✓ Fixed

Resolution
Fixed as recommended in commit 08622461a7de6160250c2eadc94efd47fd3a5a8d . The client has made this change where they considered it potentially useful.

Description

In this upgrade, several `external` functions have been split up into an `internal` part and a thin `external` wrapper that typically only calls the `internal` function and has one or two modifiers. The `internal` as well as the `external` function have been made `virtual` in such cases to allow overriding in derived contracts.

A common pattern for overriding functions is to only add a little bit of functionality and then call the same function on the parent contract. In the examples above, overriding the external function could consist of adding another modifier and then calling the original function on `super`. However, that is not possible if the function is `external` instead of `public`.

Recommendation

Generally, `external` should be preferred over `public` for functions that are not called internally. For `virtual` functions, however, `public` makes more sense if the pattern described above is considered (potentially) useful and should be allowed.

3.8 Miscellaneous Informational Points ✓ Fixed

Resolution
Fixed as recommended in commit 08622461a7de6160250c2eadc94efd47fd3a5a8d .

We collect several informational points in this finding, mostly around NatSpec annotations, events, etc.

Token Bridge:

A. The event `RemoteTokenBridgeSet` is not emitted anymore and can be removed from the interface `ITokenBridge`.

contracts/src/bridging/token/interfaces/ITokenBridge.sol:L120-L125

```
/**
 * @notice Emitted when the remote token bridge is set.
 * @param remoteTokenBridge The indexed remote token bridge address.
 * @param setBy The indexed address that set the remote token bridge.
 */
event RemoteTokenBridgeSet(address indexed remoteTokenBridge, address indexed setBy);
```

L1 Messaging:

B. `L1MessageServiceBase`: Transient storage variable `TRANSIENT_MESSAGE_SENDER` has no NatSpec annotation, and visibility is not specified explicitly (which is not wrong, but all regular state variables have a visibility specifier).

contracts/src/messaging/l1/L1MessageServiceBase.sol:L25

```
address transient TRANSIENT_MESSAGE_SENDER;
```

C. In the following two comments, the outdated reference to `MESSAGE_SENDER_TRANSIENT_KEY` should be replaced with a reference to `TRANSIENT_MESSAGE_SENDER`.

contracts/src/messaging/l1/L1MessageServiceBase.sol:L30

```
/// @dev DEPRECATED in favor of new transient storage with `MESSAGE_SENDER_TRANSIENT_KEY` key.
```

contracts/src/messaging/l1/L1MessageServiceBase.sol:L40

```
/// @notice The default value for the message sender reset to post claiming using the MESSAGE_SENDER_TRANSIENT_KEY.
```

D. The import of `MessageHashing.sol` can be removed from `L1MessageServiceBase`:

contracts/src/messaging/l1/L1MessageServiceBase.sol:L9

```
import { MessageHashing } from "../libraries/MessageHashing.sol";
```

The `using` directive for `MessageHashing` can be removed across the entire codebase:

contracts/src/messaging/l1/L1MessageServiceBase.sol:L23

```
using MessageHashing for *;
```

contracts/src/messaging/l1/L1MessageService.sol:L25

```
using MessageHashing for *;
```

contracts/src/messaging/l1/v1/ClaimMessageV1.sol:L14

```
using MessageHashing for *;
```

contracts/src/messaging/l2/v1/L2MessageServiceV1.sol:L25

```
using MessageHashing for *;
```

using directives for other libraries can be removed as well (e.g., `SparseMerkleTreeVerifier` , `EfficientLeftRightKeccak` , etc.).

E. The interface file `IClaimMessageV1` should probably be moved from `contracts/src/messaging/interfaces/` to `contracts/src/messaging/l1/v1/interfaces/` . The only import of this file in `contracts/src/messaging/l1/v1/ClaimMessageV1.sol` has to be changed accordingly.

F. In `IClaimMessageV1` , the NatSpec annotations for the `claimMessage` parameters `_value` and `_fee` are in the wrong order, i.e., reversed compared to the function parameters. This could increase the risk of accidentally using a wrong argument order when this function is called, especially since both have the same type.

contracts/src/messaging/interfaces/IClaimMessageV1.sol:L13-L25

```
* @param _from The msg.sender calling the origin message service.
* @param _to The destination address on the destination chain.
* @param _value The value to be transferred to the destination address.
* @param _fee The message service fee on the origin chain.
* @param _feeRecipient Address that will receive the fees.
* @param _calldata The calldata used by the destination message service to call/forward to the destination contract.
* @param _nonce Unique message number.
*/
function claimMessage(
    address _from,
    address _to,
    uint256 _fee,
    uint256 _value,
```

L2 Messaging:

G. `L2MessageService` has a gap variable, while comparable contracts (deployable, not inherited from) like `TokenBridge` or `LineaRollup` don't have one.

contracts/src/messaging/l2/L2MessageService.sol:L10-L12

```
/// @dev Total contract storage is 50 slots with the gap below.
/// @dev Keep 50 free storage slots for future implementation updates to avoid storage collision.
uint256[50] private __gap_L2MessageService;
```

Note that `L2MessageServiceBase` has a gap with the same name.

H. `L2MessageServiceV1` : Transient storage variable `TRANSIENT_MESSAGE_SENDER` has no NatSpec annotation, and visibility is not specified explicitly (which is not wrong, but all regular state variables have a visibility specifier).

contracts/src/messaging/l2/v1/L2MessageServiceV1.sol:L34

```
address transient TRANSIENT_MESSAGE_SENDER;
```

I. In the following two comments, the outdated reference to `MESSAGE_SENDER_TRANSIENT_KEY` should be replaced with a reference to `TRANSIENT_MESSAGE_SENDER` .

contracts/src/messaging/l2/v1/L2MessageServiceV1.sol:L39

```
/// @notice The default value for the message sender reset to post claiming using the MESSAGE_SENDER_TRANSIENT_KEY.
```

contracts/src/messaging/l2/v1/L2MessageServiceV1.sol:L42

```
/// @dev DEPRECATED in favor of new transient storage with `MESSAGE_SENDER_TRANSIENT_KEY` key.
```

J. The external view function `sender` has been made virtual in the L1 counterpart (`L1MessageService.sol`), but not in `L2MessageServiceV1` .

contracts/src/messaging/l2/v1/L2MessageServiceV1.sol:L204

```
function sender() external view returns (address originalSender) {
```

Pausing:

K. A new event `UnPausedDueToExpiry` is introduced in `IPauseManager` . The `PauseType` parameter should probably be `indexed` , as in all other events in this interface.

contracts/src/security/pausing/interfaces/IPauseManager.sol:L64

```
event UnPausedDueToExpiry(PauseType pauseType);
```

L. A comment in `PauseManager` was changed and now says number of storage slots used by the contract (incl. gap) is "12", but "11" is (and was) correct.

contracts/src/security/pausing/PauseManager.sol:L47

```
/// @dev Total contract storage is 12 slots with the gap below.
```

Rollup:

M. In `ILineaRollup` , the NatSpec annotation for the `baseInitializationData` struct member is wrong (left from `initialStateRootHash` member in the former initialization struct).

contracts/src/rollup/interfaces/ILineaRollup.sol:L13

```
* @param baseInitializationData The initial state root hash at initialization used for proof verification.
```

N. A comment in `LineaRollupBase` says that the total contract storage is 61 slots, but it is 62.

contracts/src/rollup/LineaRollupBase.sol:L99

```
/// @dev Total contract storage is 61 slots.
```

O. The `__LineaRollup_init` function in `LineaRollupBase` takes as input the initialization data and the genesis shnarf. At the end, it emits the `LineaRollupBaseInitialized` event, but this event doesn't contain the genesis shnarf.

contracts/src/rollup/LineaRollupBase.sol:L151

```
emit LineaRollupBaseInitialized(_initializationData);
```

P. Function `_computePublicInput` in `LineaRollupBase` has no NatSpec for `_lastFinalizedShnarf` parameter.

contracts/src/rollup/LineaRollupBase.sol:L419-L429

```
* @param _finalizationData The full finalization data.
* @param _finalShnarf The final shnarf in the finalization.
* @param _lastFinalizedBlockNumber The last finalized block number.
* @param _verifierChainConfiguration The verifier chain configuration.
*/
function _computePublicInput(
    FinalizationDataV3 calldata _finalizationData,
    bytes32 _lastFinalizedShnarf,
    bytes32 _finalShnarf,
    uint256 _lastFinalizedBlockNumber,
    bytes32 _verifierChainConfiguration
```

Q. The mapping `blobShnarfExists` has been renamed to `_blobShnarfExists` . Several comments in `LineaRollupBase` still mention the old name without underscore:

contracts/src/rollup/LineaRollupBase.sol:L60-L69

```
/// @dev DEPRECATED in favor of the single blobShnarfExists mapping.
mapping(bytes32 dataHash => bytes32 finalStateRootHash) private dataFinalStateRootHashes_DEPRECATED;
/// @dev DEPRECATED in favor of the single blobShnarfExists mapping.
mapping(bytes32 dataHash => bytes32 parentHash) private dataParents_DEPRECATED;
/// @dev DEPRECATED in favor of the single blobShnarfExists mapping.
mapping(bytes32 dataHash => bytes32 shnarfHash) private dataShnarfHashes_DEPRECATED;
/// @dev DEPRECATED in favor of the single blobShnarfExists mapping.
mapping(bytes32 dataHash => uint256 startingBlock) private dataStartingBlock_DEPRECATED;
/// @dev DEPRECATED in favor of the single blobShnarfExists mapping.
mapping(bytes32 dataHash => uint256 endingBlock) private dataEndingBlock_DEPRECATED;
```

R. Missing event emission `LineaRollupVersionChanged` in `reinitializeV8` function. (Self-reported by client.)

Liveness Recovery:

S. Function `setLivenessRecoveryOperator` in `LivenessRecovery` : `_grantRole` call and event emission can be skipped if the role has already been granted. (Self-reported by client.)

contracts/src/rollup/LivenessRecovery.sol:L52-L53

```
_grantRole(OPERATOR_ROLE, livenessRecoveryOperatorAddress);
emit LivenessRecoveryOperatorRoleGranted(msg.sender, livenessRecoveryOperatorAddress);
```

Data Availability:

T. Wrong `@title` NatSpec in `ShnarfDataAcceptor` .

contracts/src/rollup/dataAvailability/ShnarfDataAcceptor.sol:L8

```
* @title Contract to manage cross-chain messaging on L1, L2 data submission, and rollup proof verification.
```

Appendix 1 - Files in Scope

A.1.1 Per Phase

Phase 1: Contract Folder Restructure

If files were moved in this phase and then changed later *without being in scope in at least one of the later phases*, then these changes are out of scope. In other words, changes after the relocation must be explicitly included in scope 2, 3, or 4 for this file to be considered in scope for the final commit hash.

Old Path	New Path
contracts/contracts/tokenBridge/BridgedToken.sol	contracts/src/bridging/token/BridgedToken.sol
contracts/contracts/tokenBridge/CustomBridgedToken.sol	contracts/src/bridging/token/CustomBridgedToken.sol
contracts/contracts/tokenBridge/TokenBridge.sol	contracts/src/bridging/token/TokenBridge.sol
contracts/contracts/tokenBridge/interfaces/ITokenBridge.sol	contracts/src/bridging/token/interfaces/ITokenBridge.sol
contracts/contracts/tokenBridge/lib/StorageFiller39.sol	contracts/src/bridging/token/utils/StorageFiller39.sol
contracts/contracts/messageService/lib/TimeLock.sol	contracts/src/governance/TimeLock.sol
contracts/contracts/interfaces/IGenericErrors.sol	contracts/src/interfaces/IGenericErrors.sol
contracts/contracts/lib/Utils.sol	contracts/src/libraries/EfficientLeftRightKeccak.sol
contracts/contracts/messageService/lib/TransientStorageHelpers.sol	contracts/src/libraries/TransientStorageHelpers.sol
contracts/contracts/messageService/MessageServiceBase.sol	contracts/src/messaging/MessageServiceBase.sol
contracts/contracts/interfaces/IMessageService.sol	contracts/src/messaging/interfaces/IMessageService.sol
contracts/contracts/messageService/l1/L1MessageManager.sol	contracts/src/messaging/l1/L1MessageManager.sol
contracts/contracts/messageService/l1/L1MessageService.sol	contracts/src/messaging/l1/L1MessageService.sol
contracts/contracts/interfaces/l1/IL1MessageManager.sol	contracts/src/messaging/l1/interfaces/IL1MessageManager.sol
contracts/contracts/interfaces/l1/IL1MessageService.sol	contracts/src/messaging/l1/interfaces/IL1MessageService.sol
contracts/contracts/messageService/l1/v1/L1MessageManagerV1.sol	contracts/src/messaging/l1/v1/L1MessageManagerV1.sol
contracts/contracts/messageService/l1/v1/L1MessageServiceV1.sol	contracts/src/messaging/l1/v1/L1MessageServiceV1.sol
contracts/contracts/interfaces/l1/IL1MessageManagerV1.sol	contracts/src/messaging/l1/v1/interfaces/IL1MessageManagerV1.sol
contracts/contracts/messageService/l2/L2MessageManager.sol	contracts/src/messaging/l2/L2MessageManager.sol
contracts/contracts/messageService/l2/L2MessageService.sol	contracts/src/messaging/l2/L2MessageService.sol
contracts/contracts/interfaces/l2/IL2MessageManager.sol	contracts/src/messaging/l2/interfaces/IL2MessageManager.sol
contracts/contracts/messageService/l2/v1/L2MessageManagerV1.sol	contracts/src/messaging/l2/v1/L2MessageManagerV1.sol
contracts/contracts/messageService/l2/v1/L2MessageServiceV1.sol	contracts/src/messaging/l2/v1/L2MessageServiceV1.sol
contracts/contracts/interfaces/l2/IL2MessageManagerV1.sol	contracts/src/messaging/l2/v1/interfaces/IL2MessageManagerV1.sol
contracts/contracts/interfaces/l2/IL2MessageServiceV1.sol	contracts/src/messaging/l2/v1/interfaces/IL2MessageServiceV1.sol
contracts/contracts/messageService/lib/MessageHashing.sol	contracts/src/messaging/libraries/MessageHashing.sol
contracts/contracts/messageService/lib/SparseMerkleTreeVerifier.sol	contracts/src/messaging/libraries/SparseMerkleTreeVerifier.sol
contracts/contracts/lib/CallForwardingProxy.sol	contracts/src/proxies/CallForwardingProxy.sol
contracts/contracts/LineaRollup.sol	contracts/src/rollup/LineaRollup.sol
contracts/contracts/ZkEvmV2.sol	contracts/src/rollup/ZkEvmV2.sol
contracts/contracts/interfaces/l1/ILineaRollup.sol	contracts/src/rollup/interfaces/ILineaRollup.sol
contracts/contracts/interfaces/l1/IZkEvmV2.sol	contracts/src/rollup/interfaces/IZkEvmV2.sol
contracts/contracts/lib/PermissionsManager.sol	contracts/src/security/access/PermissionsManager.sol
contracts/contracts/interfaces/IPermissionsManager.sol	contracts/src/security/access/interfaces/IPermissionsManager.sol
contracts/contracts/messageService/lib/RateLimiter.sol	contracts/src/security/limiting/RateLimiter.sol
contracts/contracts/interfaces/IRateLimiter.sol	contracts/src/security/limiting/interfaces/IRateLimiter.sol
contracts/contracts/lib/L2MessageServicePauseManager.sol	contracts/src/security/pausing/L2MessageServicePauseManager.sol
contracts/contracts/lib/LineaRollupPauseManager.sol	contracts/src/security/pausing/LineaRollupPauseManager.sol
contracts/contracts/lib/PauseManager.sol	contracts/src/security/pausing/PauseManager.sol
contracts/contracts/lib/TokenBridgePauseManager.sol	contracts/src/security/pausing/TokenBridgePauseManager.sol

Old Path	New Path
contracts/contracts/interfaces/IPauseManager.sol	contracts/src/security/pausing/interfaces/IPauseManager.sol
contracts/contracts/messageService/l1/TransientStorageReentrancyGuardUpgradeable.sol	contracts/src/security/reentrancy/TransientStorageReentrancyGuardUpgradeable.sol
contracts/contracts/verifiers/PlonkVerifierDev.sol	contracts/src/verifiers/PlonkVerifierDev.sol
contracts/contracts/verifiers/PlonkVerifierForDataAggregation.sol	contracts/src/verifiers/PlonkVerifierForDataAggregation.sol
contracts/contracts/verifiers/PlonkVerifierForMultiTypeDataAggregation.sol	contracts/src/verifiers/PlonkVerifierForMultiTypeDataAggregation.sol
contracts/contracts/interfaces/l1/IPlonkVerifier.sol	contracts/src/verifiers/interfaces/IPlonkVerifier.sol

Phase 2: Core Feature Integration

File Path
contracts/src/bridging/token/BridgedToken.sol
contracts/src/bridging/token/CustomBridgedToken.sol
contracts/src/bridging/token/TokenBridge.sol
contracts/src/bridging/token/TokenBridgeBase.sol
contracts/src/bridging/token/interfaces/ITokenBridge.sol
contracts/src/bridging/token/utils/StorageFiller39.sol
contracts/src/governance/TimeLock.sol
contracts/src/interfaces/IGenericErrors.sol
contracts/src/libraries/EfficientLeftRightKeccak.sol
contracts/src/libraries/TransientStorageHelpers.sol
contracts/src/messaging/MessageServiceBase.sol
contracts/src/messaging/interfaces/IMessageService.sol
contracts/src/messaging/l1/L1MessageManager.sol
contracts/src/messaging/l1/L1MessageService.sol
contracts/src/messaging/l1/interfaces/IL1MessageManager.sol
contracts/src/messaging/l1/interfaces/IL1MessageService.sol
contracts/src/messaging/l1/v1/L1MessageManagerV1.sol
contracts/src/messaging/l1/v1/L1MessageServiceV1.sol
contracts/src/messaging/l1/v1/interfaces/IL1MessageManagerV1.sol
contracts/src/messaging/l2/L2MessageManager.sol
contracts/src/messaging/l2/L2MessageService.sol
contracts/src/messaging/l2/L2MessageServiceBase.sol
contracts/src/messaging/l2/interfaces/IL2MessageManager.sol
contracts/src/messaging/l2/v1/L2MessageManagerV1.sol
contracts/src/messaging/l2/v1/L2MessageServiceV1.sol
contracts/src/messaging/l2/v1/interfaces/IL2MessageManagerV1.sol
contracts/src/messaging/l2/v1/interfaces/IL2MessageServiceV1.sol
contracts/src/messaging/libraries/MessageHashing.sol
contracts/src/messaging/libraries/SparseMerkleTreeVerifier.sol
contracts/src/proxies/CallForwardingProxy.sol
contracts/src/rollup/LineaRollup.sol
contracts/src/rollup/LineaRollupBase.sol
contracts/src/rollup/ZkEvmV2.sol
contracts/src/rollup/interfaces/ILineaRollup.sol
contracts/src/rollup/interfaces/IZkEvmV2.sol
contracts/src/security/access/PermissionsManager.sol
contracts/src/security/access/interfaces/IPermissionsManager.sol
contracts/src/security/limiting/RateLimiter.sol
contracts/src/security/limiting/interfaces/IRateLimiter.sol
contracts/src/security/pausing/L2MessageServicePauseManager.sol
contracts/src/security/pausing/LineaRollupPauseManager.sol

File Path
contracts/src/security/pausing/PauseManager.sol
contracts/src/security/pausing/TokenBridgePauseManager.sol
contracts/src/security/pausing/interfaces/IPauseManager.sol
contracts/src/security/reentrancy/TransientStorageReentrancyGuardUpgradeable.sol

Phase 3: Dynamic Chain Configuration

File Path
contracts/src/rollup/LineaRollup.sol
contracts/src/rollup/LineaRollupBase.sol
contracts/src/rollup/ZkEvmV2.sol
contracts/src/verifiers/interfaces/IPlonkVerifier.sol

Phase 4: Data Availability Integration

File Path
contracts/src/rollup/LineaRollup.sol
contracts/src/rollup/LineaRollupBase.sol
contracts/src/rollup/LivenessRecovery.sol
contracts/src/rollup/Validium.sol
contracts/src/rollup/dataAvailability/Eip4844BlobAcceptor.sol
contracts/src/rollup/dataAvailability/CalldataBlobAcceptor.sol
contracts/src/rollup/dataAvailability/ShnarfDataAcceptor.sol
contracts/src/rollup/dataAvailability/LocalShnarfProvider.sol
contracts/src/messaging/l1/L1MessageService.sol
contracts/src/messaging/l1/L1MessageServiceBase.sol
contracts/src/messaging/l1/v1/ClaimMessageV1.sol

A.1.2 SHA-1 Hashes of Files in Scope for the Final Commit

The following list records the SHA-1 hashes of the files in scope at commit [08622461a7de6160250c2eadc94efd47fd3a5a8d](#), where the fixes to the issues found during this engagement have been implemented.

File	SHA-1 hash
contracts/src/bridging/token/BridgedToken.sol	e30def2fbd7b1a4b55cf9c188505ce73a81b9b92
contracts/src/bridging/token/CustomBridgedToken.sol	ce5679f64198145da4c751f2fd801209b4a7e1fb
contracts/src/bridging/token/TokenBridge.sol	73155e3ee930c77b9b0cce3da9663cca7159f2a9
contracts/src/bridging/token/TokenBridgeBase.sol	6b37d3430f1c50490af8d40bab8055184597dcb7
contracts/src/bridging/token/interfaces/ITokenBridge.sol	e7facd8b518e200d205b6a7da9835b8776ac582c
contracts/src/bridging/token/utils/StorageFiller39.sol	9356c3e722e2894a19142da54bd7955744de7923
contracts/src/common/InitializationVersionCheck.sol	458025c96b2928f02c59446ea02f709f015623d5
contracts/src/governance/TimeLock.sol	2c7d44af00e5e97f83848fd4b1a0bb85277418fe
contracts/src/interfaces/IGenericErrors.sol	d375018c303e01d33e3975b8f8871920e7eecb04
contracts/src/libraries/EfficientLeftRightKeccak.sol	253f164004e350c6f70001f9f20a479e1817c50c
contracts/src/messaging/MessageServiceBase.sol	712c074c8c75185141250cb92bc0b8bc0056fab4
contracts/src/messaging/interfaces/IMessageService.sol	483c694c3e52cbe7b17ac3e0b8c3b1f3d00e98c7
contracts/src/messaging/l1/L1MessageManager.sol	5b51d12ca06d3773bd26c8c9b08763856e73e03f
contracts/src/messaging/l1/L1MessageService.sol	4129e366afad150c370db24b04868acda508efde
contracts/src/messaging/l1/L1MessageServiceBase.sol	489fe1b2d11db7b3dc19de3f6bdf79991793c3fb
contracts/src/messaging/l1/interfaces/IL1MessageManager.sol	ba2078bb48fa9afb773d518e627e50610fd05ccb
contracts/src/messaging/l1/interfaces/IL1MessageService.sol	a6d16f3608f643cb8eb1fc594a2b7423fbf6c636
contracts/src/messaging/l1/v1/ClaimMessageV1.sol	39bc3cd95b5f3da7b64c13519dd0aff074a5e62
contracts/src/messaging/l1/v1/L1MessageManagerV1.sol	9e50679c6e927ac55f911a80c074e1c6fc57a9e7
contracts/src/messaging/l1/v1/interfaces/IClaimMessageV1.sol	a7209ac4cbe062e3d142220f80738a1ab25bb84
contracts/src/messaging/l1/v1/interfaces/IL1MessageManagerV1.sol	ce224a4eaccb02fd3bb14c6fa3739d3ab5ef6b75
contracts/src/messaging/l2/L2MessageManager.sol	a765ad5e64a17a885dce3fbfcbf91eba1f34e24e
contracts/src/messaging/l2/L2MessageService.sol	2940fc7cc8c1b01b3f05631b8a1f4913c73150ce
contracts/src/messaging/l2/L2MessageServiceBase.sol	e90f3adb63147fcdcc553d44873a6c845b7dff88

File	SHA-1 hash
contracts/src/messaging/l2/interfaces/IL2MessageManager.sol	c22da18d738b51d4468ab8718391e809e8c140cd
contracts/src/messaging/l2/v1/L2MessageManagerV1.sol	2a9b019e74743e3afdf885698223b3e0b422736
contracts/src/messaging/l2/v1/L2MessageServiceV1.sol	65b3ae1206c2c3f00c4921ea1a53882334c6eaac
contracts/src/messaging/l2/v1/interfaces/IL2MessageManagerV1.sol	bc7e6e884f1579d6384cb65cd0c0b482ad65c871
contracts/src/messaging/l2/v1/interfaces/IL2MessageServiceV1.sol	b782056e95e1cc7ba28bc4fe0170a31e6b13c6db
contracts/src/messaging/libraries/MessageHashing.sol	d0255d57d53c30d8ae60a959cc4a4f63eb03f969
contracts/src/messaging/libraries/SparseMerkleTreeVerifier.sol	4caf98b61b8675d5c2b274f75dfc5060a5a6ed1b
contracts/src/proxies/CallForwardingProxy.sol	2778ddd34f9565328c208d6ab289b7f1e9f59d9d
contracts/src/rollup/LineaRollup.sol	0e26d546fd0e4f20806e0357895dc673d26ea076
contracts/src/rollup/LineaRollupBase.sol	7131879a8db7658d3f9358a45b96a152ada4ff46
contracts/src/rollup/LivenessRecovery.sol	1237416654bff589dbb9e921c00eb6fc0518e190
contracts/src/rollup/Validium.sol	ea23514132ec6c5b68a3a74b3e5b397a838450eb
contracts/src/rollup/ZkEvmV2.sol	6de224c3742a89984e0198ca59ddbc7fb9b45c1c
contracts/src/rollup/dataAvailability/CalldataBlobAcceptor.sol	f596d2eccc819b9423b430c4556272af5bf7bc15
contracts/src/rollup/dataAvailability/Eip4844BlobAcceptor.sol	d310a4046eae83232861c25c1d57a4593f40f3d3
contracts/src/rollup/dataAvailability/LocalShnarfProvider.sol	a8997273046c7cf38dfbe25be9a0034327d82691
contracts/src/rollup/dataAvailability/ShnarfDataAcceptor.sol	f78659b8eba8de7855135c75c2204e0aa382f591
contracts/src/rollup/dataAvailability/ShnarfDataAcceptorBase.sol	5efa6a3e4c36f4f58f85dc7430307bd80fd17aa2
contracts/src/rollup/dataAvailability/interfaces/IAcceptCalldataBlobs.sol	62e26757ee125b260b54a10c5f949e0f48690df0
contracts/src/rollup/dataAvailability/interfaces/IAcceptEip4844Blobs.sol	f653b1da6def44a14fe1c9b9008f0c68fdd4c259
contracts/src/rollup/dataAvailability/interfaces/IAcceptShnarfData.sol	a622da5dc9ffe255a8bcaad2744af6716b631831
contracts/src/rollup/dataAvailability/interfaces/IProvideShnarf.sol	731c2df94b579fac8eceeaa53b22d32de9dc3604
contracts/src/rollup/dataAvailability/interfaces/IShnarfDataAcceptorBase.sol	8905967a6b6f995d9d77878ea497c85b1c025f0c
contracts/src/rollup/interfaces/ILineaRollup.sol	c5c6871cf4f3b4e8e2d32590d0a7932fdf74fd24
contracts/src/rollup/interfaces/ILineaRollupBase.sol	a3d0a285661875187aa3941d8b3a18edfe93f38a
contracts/src/rollup/interfaces/ILivenessRecovery.sol	cb815d584045a360d6207c0747fa3bfc8b0e5ad
contracts/src/rollup/interfaces/IZkEvmV2.sol	1483028ceb8edb482ee5999dcf79e56983301f9
contracts/src/security/access/PermissionsManager.sol	c00df08c6ae9984653f26aea809e991e0991d1b0
contracts/src/security/access/interfaces/IPermissionsManager.sol	dff0ae179c538bf515efade000ef64df3bbad4d
contracts/src/security/limiting/RateLimiter.sol	25a39d6b0a0a2322821713ca2891b46e46bb7150
contracts/src/security/limiting/interfaces/IRateLimiter.sol	36dc4fc10cd1c390ab6bf76643fb3e49826180a9
contracts/src/security/pausing/L2MessageServicePauseManager.sol	cac85cba9af2aa645e2c3541ef13af8647a41594
contracts/src/security/pausing/LineaRollupPauseManager.sol	d8e16fd186e98717ba43a26acbb77cc048dc863c
contracts/src/security/pausing/PauseManager.sol	51209264fec87ec50db31b2c577768850e5433e8
contracts/src/security/pausing/TokenBridgePauseManager.sol	61215a127731c2db34a8e5f6087d0405b1611f9a
contracts/src/security/pausing/interfaces/IPauseManager.sol	51070d4ec343252c69ebbb6effdf6b40f2f8bdda
contracts/src/security/reentrancy/TransientStorageReentrancyGuardUpgradeable.sol	ee820c4a433d26a3e30bbaee3032e478f2cb216b
contracts/src/verifiers/interfaces/IPlonkVerifier.sol	bd14bf68f6769fbe72923ec2a54f3b3bc228b382

Appendix 2 - Disclosure

Consensys Diligence (“CD”) typically receives compensation from one or more clients (the “Clients”) for performing the analysis contained in these reports (the “Reports”). The Reports may be distributed through other means, including via Consensys publications and other distributions.

The Reports are not an endorsement or indictment of any particular project or team, and the Reports do not guarantee the security of any particular project. This Report does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. No Report provides any warranty or representation to any third party in any respect, including regarding the bug-free nature of code, the business model or proprietors of any such business model, and the legal compliance of any such business. No third party should rely on the Reports in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset. Specifically, for the avoidance of doubt, this Report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. CD owes no duty to any third party by virtue of publishing these Reports.

A.2.1 Purpose of Reports

The Reports and the analysis described therein are created solely for Clients and published with their consent. The scope of our review is limited to a review of code and only the code we note as being within the scope of our review within this report. Any

Solidity code itself presents unique and unquantifiable risks as the Solidity language itself remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond specified code that could present security risks. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. In some instances, we may perform penetration testing or infrastructure assessments depending on the scope of the particular engagement.

CD makes the Reports available to parties other than the Clients (i.e., “third parties”) on its website. CD hopes that by making these analyses publicly available, it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.

A.2.2 Links to Other Web Sites from This Web Site

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Consensys and CD. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Consensys and CD are not responsible for the content or operation of such Web sites, and that Consensys and CD shall have no liability to you or any other person or entity for the use of third party Web sites. Except as described below, a hyperlink from this web Site to another web site does not imply or mean that Consensys and CD endorses the content on that Web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the Reports. Consensys and CD assumes no responsibility for the use of third-party software on the Web Site and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.

A.2.3 Timeliness of Content

The content contained in the Reports is current as of the date appearing on the Report and is subject to change without notice unless indicated otherwise, by Consensys and CD.